

工程框架

大型工程中的工程框架是十分重要的，框架搭的好，代码就会十分易读。笔者这里介绍自己常用的工程框架搭建方法。

1. 分层架构

分层架构对于嵌入式工程是必要的。常见的分层架构体系如下：



通常要实现**算法层**，**功能模块层**和**应用程序逻辑层**。以笔者自己设计的 FOC 电机控制台架代码为例，其文件架构如下：

```
1 FOC_Project
2 |─ Usercode // 应用程序逻辑层，实现FOC算法框架
3 | |─ drivefoc.c
4 | |─ drivefoc.h
5 | |─ loadfoc.c
6 | |─ loadfoc.h
7 | |─ userinit.c
8 | |─ userinit.h
9 | |─ usermain.c
10 | |─ usermain.h
11 | |─ userpara.c
12 | |─ userpara.h
13 | |─ user_transfunc.c
14 | |─ user_transfunc.h
15 |─ HFI // 应用程序逻辑层
16 | |─ square_hfi.c
17 | |─ square_hfi.h
18 |─ Hardwarecode // 应用程序逻辑层，实现其他小型功能
19 | |─ hwdac.c
20 | |─ hwdac.h
21 | |─ hwinit.c
22 | |─ hwinit.h
23 | |─ hwit.c
24 | |─ hwit.h
25 | |─ hwprint.c
26 |─ FOC // 算法层
27 | |─ coordinate_transform.c
28 | |─ coordinate_transform.h
29 | |─ curr_sample.c
30 | |─ curr_sample.h
31 | |─ foc_math.c
32 | |─ foc_math.h
33 | |─ svpwm.c
34 | |─ svpwm.h
```

```
35 | └ Encoder                                     // 功能模块层
36 | | └ ad2s1210.c
37 | | └ ad2s1210.h
38 | └ Core                                     // (系统生成) 硬件抽象层
39 | | └ Src
40 | | | └ adc.c
41 | | | └ dac.c
42 | | | └ gpio.c
43 | | | └ main.c
44 | | | └ memorymap.c
45 | | | └ spi.c
46 | | | └ stm32h7xx_hal_msp.c
47 | | | └ stm32h7xx_it.c
48 | | | └ system_stm32h7xx.c
49 | | | └ tim.c
50 | | └ └ usart.c
51 | | └ Inc
52 | | | └ adc.h
53 | | | └ dac.h
54 | | | └ gpio.h
55 | | | └ main.h
56 | | | └ memorymap.h
57 | | | └ spi.h
58 | | | └ stm32h7xx_hal_conf.h
59 | | | └ stm32h7xx_it.h
60 | | | └ tim.h
61 | | └ └ usart.h
```

对于算法层和功能模块层，要点在于**不应当设置全局变量**，应当设置对应算法/功能模块的**数据类型(数据)**和**调用方法**。

比如 MT6701 编码器控制文件：

```
1 // mt6701.h
2 /**
3  * @file      STM32 MT6701 磁编码器驱动程序
4  * @brief     MT6701 是 14Bit 的单圈绝对角度检测传感器,拥有 ABZ/PWM/模拟量/I2C/SPI 等多种信息输出方式,这里采用 IIC
   进行数据输出
5  * @attention 使用前请选择驱动方式(硬件/软件IIC)
6  *           使用时使用MT6701_Init()进行初始化
7  *           在一个循环的程序中定时运行MT6701_Get_Angle()
8  * @author   SSC
9  */
10 #ifndef __MT6701_H
11 #define __MT6701_H
12
13 #include "stm32g4xx.h"
14
15 /***** 驱动方式选择 *****/
16
17 #define MT6701_SoftWare_IIC 1 // 使用软件IIC
18 #define MT6701_HardWare_IIC 0 // 使用硬件IIC
19
20 /***** 端口定义 *****/
21
22 #if (MT6701_SoftWare_IIC == 1)
23
24 #define MT6701_SCL_Port GPIOA
25 #define MT6701_SDA_Port GPIOB
26 #define MT6701_SCL_Pin  GPIO_PIN_15
27 #define MT6701_SDA_Pin  GPIO_PIN_7
28
29 #endif
30
31 /***** 寄存器定义 *****/
32
33 #define MT6701_ANGLE_REG 0x03 // 角度寄存器
```

```

34
35 #define MT6701_ADDR      0x06 // 设备地址
36
37 /***** 函数定义 *****/
38 void MT6701_Get_Angle(float *encoder_angle);
39
40 #endif

```

```

1 // mt6701.c
2 #include "mt6701.h"
3
4 /*****宏定义部分*****/
5
6 /* 如果使能软件IIC */
7 #if (MT6701_SoftWare_IIC == 1)
8 #include <inttypes.h>
9 #include "gpio.h"
10 #define IIC_WR 0 /* 写控制bit */
11 #define IIC_RD 1 /* 读控制bit */
12 #define IIC_SCL(x)
\
13     do {
\
14         (x == 1) ? HAL_GPIO_WritePin(MT6701_SCL_Port, MT6701_SCL_Pin, GPIO_PIN_SET) :
HAL_GPIO_WritePin(MT6701_SCL_Port, MT6701_SCL_Pin, GPIO_PIN_RESET); \
15     } while (0)
16
17 #define IIC_SDA(x)
\
18     do {
\
19         (x == 1) ? HAL_GPIO_WritePin(MT6701_SDA_Port, MT6701_SDA_Pin, GPIO_PIN_SET) :
HAL_GPIO_WritePin(MT6701_SDA_Port, MT6701_SDA_Pin, GPIO_PIN_RESET); \
20     } while (0)
21
22 #define IIC_SDA_READ() HAL_GPIO_ReadPin(MT6701_SDA_Port, MT6701_SDA_Pin) /* 读SDA口线状态 */
23 #endif
24
25 /* 如果使能硬件IIC */
26 #if (MT6701_HardWare_IIC == 1)
27
28 #include "i2c.h"
29 #define MT6701_WRITE_ADDR (0x0C) // MT6701 写地址
30 #define MT6701_READ_ADDR (0x0D) // MT6701 读地址
31
32 #endif
33
34 /*****IIC基础驱动函数*****/
35
36 /* 如果使能软件IIC */
37
38 #if (MT6701_SoftWare_IIC == 1)
39 /**
40  * @brief      IIC 总线延迟, 最快为400kHz
41  * @attention  循环次数为10时, SCL频率 = 205KHz
42                循环次数为7时, SCL频率 = 347KHz,  SCL高电平时间1.5us, SCL低电平时间2.87us
43                循环次数为5时, SCL频率 = 421KHz,  SCL高电平时间1.25us, SCL低电平时间2.375us
44  */
45 static void _IIC_Delay(void)
46 {
47     uint8_t i;
48
49     for (i = 0; i < 10; i++);
50 }
51

```

```

52  /**
53   * @brief      CPU发起IIC总线启动信号
54   * @attention   当SCL高电平时，SDA出现一个下跳沿表示IIC总线启动信号
55   */
56  static void _IIC_Start(void)
57  {
58      IIC_SDA(1);
59      IIC_SCL(1);
60      _IIC_Delay();
61      IIC_SDA(0);
62      _IIC_Delay();
63      IIC_SCL(0);
64      _IIC_Delay();
65  }
66
67  /**
68   * @brief      CPU发起IIC总线停止信号
69   * @attention   当SCL高电平时，SDA出现一个上跳沿表示IIC总线停止信号
70   */
71  static void _IIC_Stop(void)
72  {
73      IIC_SDA(0);
74      IIC_SCL(1);
75      _IIC_Delay();
76      IIC_SDA(1);
77  }
78
79  /**
80   * @brief      CPU向IIC总线设备发送8bit数据
81   * @param      byte    发送的8字节数据
82   */
83  static void _IIC_Send_Byte(uint8_t byte)
84  {
85      uint8_t i;
86
87      /* 先发送字节的高位bit7 */
88      for (i = 0; i < 8; i++) {
89          if (byte & 0x80) {
90              IIC_SDA(1);
91          } else {
92              IIC_SDA(0);
93          }
94          _IIC_Delay();
95          IIC_SCL(1);
96          _IIC_Delay();
97          IIC_SCL(0);
98          if (i == 7) {
99              IIC_SDA(1); // 释放总线
100          }
101          byte <<= 1; // 左移一位
102          _IIC_Delay();
103      }
104  }
105
106  /**
107   * @brief      CPU产生一个ACK信号
108   */
109  static void _IIC_Ack(void)
110  {
111      IIC_SDA(0); /* CPU驱动SDA = 0 */
112      _IIC_Delay();
113      IIC_SCL(1); /* CPU产生1个时钟 */
114      _IIC_Delay();
115      IIC_SCL(0);
116      _IIC_Delay();

```

```

117     IIC_SDA(1); /* CPU释放SDA总线 */
118 }
119
120 /**
121  * @brief    CPU产生1个NACK信号
122  */
123 static void _IIC_NAck(void)
124 {
125     IIC_SDA(1); /* CPU驱动SDA = 1 */
126     _IIC_Delay();
127     IIC_SCL(1); /* CPU产生1个时钟 */
128     _IIC_Delay();
129     IIC_SCL(0);
130     _IIC_Delay();
131 }
132
133 /**
134  * @brief    CPU从IIC总线设备读取8bit数据
135  * @return    读取的数据
136  */
137 static uint8_t _IIC_Read_Byte(uint8_t ack)
138 {
139     uint8_t i;
140     uint8_t value;
141
142     /* 读到第1个bit为数据的bit7 */
143     value = 0;
144     for (i = 0; i < 8; i++) {
145         value <<= 1;
146         IIC_SCL(1);
147         _IIC_Delay();
148         if (IIC_SDA_READ()) {
149             value++;
150         }
151         IIC_SCL(0);
152         _IIC_Delay();
153     }
154     if (ack == 0)
155         _IIC_NAck();
156     else
157         _IIC_Ack();
158     return value;
159 }
160
161 /**
162  * @brief    CPU产生一个时钟，并读取器件的ACK应答信号
163  * @return    返回0表示正确应答，1表示无器件响应
164  */
165 static uint8_t _IIC_Wait_Ack(void)
166 {
167     uint8_t re;
168
169     IIC_SDA(1); /* CPU释放SDA总线 */
170     _IIC_Delay();
171     IIC_SCL(1); /* CPU驱动SCL = 1，此时器件会返回ACK应答 */
172     _IIC_Delay();
173     if (IIC_SDA_READ()) /* CPU读取SDA口线状态 */
174     {
175         re = 1;
176     } else {
177         re = 0;
178     }
179     IIC_SCL(0);
180     _IIC_Delay();
181     return re;

```

```

182 }
183
184 /**
185  * @brief    配置IIC总线的GPIO，采用模拟IO的方式实现
186  * @attention 在CubeMX里实现，选择高速开漏输出
187  */
188 void _IIC_GPIO_Init(void)
189 {
190     MX_GPIO_Init();
191     _IIC_Stop();
192 }
193
194 /**
195  * @brief    检测IIC总线设备，CPU向发送设备地址，然后读取设备应答来判断该设备是否存在
196  * @param    _Address 设备的IIC总线地址
197  * @return    0表示正确,1表示未探测到
198  */
199 static uint8_t _IIC_CheckDevice(uint8_t _Address)
200 {
201     uint8_t ucAck;
202
203     _IIC_GPIO_Init();          /* 配置GPIO */
204     _IIC_Start();              /* 发送启动信号 */
205     _IIC_Send_Byte(_Address | IIC_WR); /* 发送设备地址+读写控制bit (0 = w, 1 = r) bit7 先传 */
206     ucAck = _IIC_Wait_Ack();    /* 检测设备的ACK应答 */
207     _IIC_Stop();               /* 发送停止信号 */
208
209     return ucAck;
210 }
211
212 /**
213  * @brief    IIC连续写
214  * @param    addr    器件地址
215  * @param    reg      寄存器地址
216  * @param    len      写入长度
217  * @param    buf      数据区
218  */
219 uint8_t MT6701_Write_Len(uint8_t addr, uint8_t reg, uint8_t len, uint8_t *buf)
220 {
221     uint8_t i;
222     _IIC_Start();
223     _IIC_Send_Byte((addr << 1) | 0); // 发送器件地址+写命令
224     if (_IIC_Wait_Ack())              // 等待应答
225     {
226         _IIC_Stop();
227         return 1;
228     }
229     _IIC_Send_Byte(reg); // 写寄存器地址
230     _IIC_Wait_Ack();      // 等待应答
231     for (i = 0; i < len; i++) {
232         _IIC_Send_Byte(buf[i]); // 发送数据
233         if (_IIC_Wait_Ack())     // 等待ACK
234         {
235             _IIC_Stop();
236             return 1;
237         }
238     }
239     _IIC_Stop();
240     return 0;
241 }
242
243 /**
244  * @brief    IIC连续读
245  * @param    addr    器件地址
246  * @param    reg      寄存器地址

```

```

247  * @param   len       写入长度
248  * @param   buf       数据区
249  */
250  uint8_t MT6701_Read_Len(uint8_t addr, uint8_t reg, uint8_t len, uint8_t *buf)
251  {
252      _IIC_Start();
253      _IIC_Send_Byte((addr << 1) | 0); // 发送器件地址+写命令
254      if (_IIC_Wait_Ack())              // 等待应答
255      {
256          _IIC_Stop();
257          return 1;
258      }
259      _IIC_Send_Byte(reg); // 写寄存器地址
260      _IIC_Wait_Ack();      // 等待应答
261      _IIC_Start();
262      _IIC_Send_Byte((addr << 1) | 1); // 发送器件地址+读命令
263      _IIC_Wait_Ack();              // 等待应答
264      while (len) {
265          if (len == 1)
266              *buf = _IIC_Read_Byte(0); // 读数据,发送nACK
267          else
268              *buf = _IIC_Read_Byte(1); // 读数据,发送ACK
269          len--;
270          buf++;
271      }
272      _IIC_Stop(); // 产生一个停止条件
273      return 0;
274  }
275
276  /**
277  * @brief   IIC写字节
278  * @param   reg       寄存器地址
279  * @param   data      数据
280  */
281  static uint8_t MT6701_Write_Byte(uint8_t reg, uint8_t data)
282  {
283      _IIC_Start();
284      _IIC_Send_Byte((MT6701_ADDR << 1) | 0); // 发送器件地址+写命令
285      if (_IIC_Wait_Ack())              // 等待应答
286      {
287          _IIC_Stop();
288          return 1;
289      }
290      _IIC_Send_Byte(reg); // 写寄存器地址
291      _IIC_Wait_Ack();      // 等待应答
292      _IIC_Send_Byte(data); // 发送数据
293      if (_IIC_Wait_Ack()) // 等待ACK
294      {
295          _IIC_Stop();
296          return 1;
297      }
298      _IIC_Stop();
299      return 0;
300  }
301
302  /**
303  * @brief   IIC读字节
304  * @param   reg       寄存器地址
305  */
306  static uint8_t MT6701_Read_Byte(uint8_t reg)
307  {
308      uint8_t res;
309      _IIC_Start();
310      _IIC_Send_Byte((MT6701_ADDR << 1) | 0); // 发送器件地址+写命令
311      _IIC_Wait_Ack();              // 等待应答

```

```

312     _IIC_Send_Byte(reg);                // 写寄存器地址
313     _IIC_Wait_Ack();                    // 等待应答
314     _IIC_Start();
315     _IIC_Send_Byte((MT6701_ADDR << 1) | 1); // 发送器件地址+读命令
316     _IIC_Wait_Ack();                    // 等待应答
317     res = _IIC_Read_Byte(0);             // 读取数据,发送nACK
318     _IIC_Stop();                        // 产生一个停止条件
319     return res;
320 }
321
322 #endif
323
324 /* 如果使能硬件IIC */
325 #if (MT6701_HardWare_IIC == 1)
326
327 /**
328  * @brief   MT6701 向寄存器内写字节
329  * @param   reg       寄存器地址
330  * @param   value     数据
331  */
332 static uint8_t MT6701_Write_Byte(uint8_t reg, uint8_t data)
333 {
334     return HAL_I2C_Mem_Write(&hi2c1, MT6701_WRITE_ADDR, reg, I2C_MEMADD_SIZE_8BIT, &data, 1, 50);
335 }
336
337 /**
338  * @brief   MT6701 向寄存器内连续写
339  * @param   reg       寄存器地址
340  * @param   value     数据
341  * @param   len       数据长度
342  */
343 static uint8_t MT6701_Write_Len(uint8_t addr, uint8_t reg, uint8_t len, uint8_t *buf)
344 {
345     return HAL_I2C_Mem_Write(&hi2c1, MT6701_WRITE_ADDR, reg, I2C_MEMADD_SIZE_8BIT, buf, len, 50);
346 }
347
348 /**
349  * @brief   MT6701 从寄存器内连续读
350  * @param   reg       寄存器地址
351  * @param   buf       数据
352  * @param   len       数据长度
353  */
354 static uint8_t MT6701_Read_Len(uint8_t addr, uint8_t reg, uint8_t len, uint8_t *buf)
355 {
356     return HAL_I2C_Mem_Read(&hi2c1, MT6701_READ_ADDR, reg, I2C_MEMADD_SIZE_8BIT, buf, len, 50);
357 }
358
359 #endif
360
361 /*****用户函数*****/
362
363 /**
364  * @brief   MT6701 初始化函数
365  */
366 void MT6701_Init(void)
367 {
368     #if (MT6701_SoftWare_IIC == 1)
369         _IIC_GPIO_Init(); // 初始化IIC总线
370     #endif
371 }
372
373 /**
374  * @brief   MT6701 读取角度
375  * @param   angle    绝对角度值
376  */

```

```

377 void MT6701_Get_Angle(float *angle)
378 {
379     uint8_t temp[2];
380     int16_t encoder_angle;
381     MT6701_Read_Len(MT6701_ADDR, MT6701_ANGLE_REG, 2, temp);
382
383     encoder_angle = ((int16_t)temp[0] << 6) | (temp[1] >> 2);
384     *angle        = (float)encoder_angle * (2 * 3.141592654f) / 16384.f;
385 }

```

或者控制算法文件:

```

1 // user_transfunc.h
2 #ifndef __USER_TRANSFUNC_H
3 #define __USER_TRANSFUNC_H
4
5 #include "foc_math.h"
6
7 /**
8  * PID 相关定义
9  */
10 typedef struct {
11     float KP;          // PID参数P
12     float KI;          // PID参数I
13     float KD;          // PID参数D
14     float fdb;         // PID反馈值
15     float ref;         // PID目标值
16     float cur_error;   // 当前误差
17     float error[2];    // 前两次误差
18     float output;      // 输出值
19     float outputMax;   // 最大输出值的绝对值
20     float outputMin;   // 最小输出值的绝对值用于防抖
21 } PID_t;
22
23 /**
24  * IIR 滤波器相关定义
25  */
26 typedef struct
27 {
28     float tsample;     // 采样时间
29     float wc;          // 截止频率
30     float alpha;       // 滤波器系数
31     float input;       // 当前输入值
32     float output_last; // 前次输出值
33     float output;      // 当前输出值
34 } LPF_t;
35
36 void PID_init(PID_t *pid, float kp, float ki, float kd, float outputMax);
37 void PID_Calc(PID_t *pid, uint8_t enable, float t_sample);
38
39 void LPF_Init(LPF_t *lpf, float f_c, float t_sample);
40 void LPF_Calc(LPF_t *lpf, uint8_t enable);
41
42 #endif

```

```

1 // user_transfunc.c
2 #include "user_transfunc.h"
3
4 /***** PID 控制器 *****/
5
6 /**
7  * @brief    增量式 PID 控制器初始化
8  * @param    pid        PID 控制器句柄
9  * @param    kp        Kp 比例参数
10  * @param    ki        Ki 积分参数(考虑采样时间)

```

```

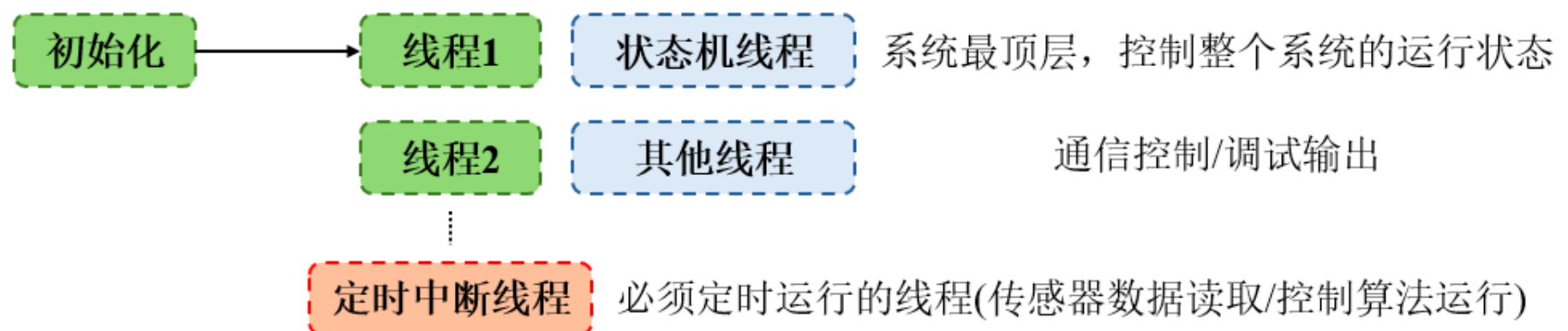
11  * @param   kd           Kd 微分参数
12  * @param   outputMax    输出饱和值
13  */
14  void PID_init(PID_t *pid, float kp, float ki, float kd, float outputMax)
15  {
16      pid->KP          = kp;
17      pid->KI          = ki;
18      pid->KD          = kd;
19      pid->outputMax = outputMax;
20  }
21
22  /**
23   * @brief    增量式PID计算
24   * @param   pid          PID 控制器句柄
25   * @param   enable        使能信号    若此值为0,PID控制器的输入,反馈和输出都置为0
26   * @param   t_sample      采样时间
27   */
28  void PID_Calc(PID_t *pid, uint8_t enable, float t_sample)
29  {
30      if (enable == 1) {
31          pid->cur_error = pid->ref - pid->fdb;
32          pid->output += pid->KP * (pid->cur_error - pid->error[1]) + pid->KI * t_sample * pid->cur_error + pid->KD * (pid->cur_error - 2 * pid->error[1] + pid->error[0]);
33          pid->error[0] = pid->error[1];
34          pid->error[1] = pid->ref - pid->fdb;
35          /*设定输出上限*/
36          if (pid->output > pid->outputMax) pid->output = pid->outputMax;
37          if (pid->output < -pid->outputMax) pid->output = -pid->outputMax;
38      } else {
39          pid->ref          = 0;
40          pid->fdb          = 0;
41          pid->output       = 0;
42          pid->cur_error     = 0;
43          pid->error[0]     = 0;
44          pid->error[1]     = 0;
45      }
46  }
47
48  /***** IIR 滤波器 *****/
49
50  /**
51   * @brief    数字一阶低通滤波器初始化(后向差分法)
52   * @param   lpf          滤波器句柄
53   * @param   f_c          截止频率
54   * @param   t_sample      采样时间
55   */
56  void LPF_Init(LPF_t *lpf, float f_c, float t_sample)
57  {
58      lpf->wc          = 2 * M_PI * f_c;
59      lpf->tsample      = t_sample;
60      lpf->alpha        = (lpf->wc * lpf->tsample) / ((lpf->wc * lpf->tsample) + 1.0f);
61      lpf->output_last = 0;
62      lpf->output       = 0;
63      lpf->input        = 0;
64  }
65
66  /**
67   * @brief    数字一阶低通滤波器计算(后向差分法)
68   * @param   lpf          滤波器句柄
69   * @param   enable        滤波器使能参数
70   */
71  void LPF_Calc(LPF_t *lpf, uint8_t enable)
72  {
73      if (enable == 1) {
74          lpf->output = lpf->output_last + lpf->alpha * (lpf->input - lpf->output_last);

```

```
75     lpf->output_last = lpf->output;
76 } else {
77     lpf->output      = 0;
78     lpf->output_last = 0;
79     lpf->input       = 0;
80 }
81 }
```

层内调用函数(层外不调用)应当内联(使用 `inline` 关键字/ `static` 关键字)。

2. 应用程序逻辑的实现



系统需要一个状态机控制整体的运行状态，同时必要的采样/控制算法需要在定时器中断内运行。其他线程(如通讯控制/屏幕显示/调试输出print)可以在主线程内(裸机)，也可以放在主线程外(操作系统)。

对应的文件架构如下(以 FOC 电机台架为例)：

```
1 // usermain.c
2
3 #include "usermain.h"
4
5 /**
6  * @brief   Init Program
7  */
8 static void init(void)
9 {
10     // User init
11     user_init();
12     // Hardware init
13     hw_init();
14 }
15
16 /**
17  * @brief   主函数
18  */
19 void usermain(void)
20 {
21     init();
22     while (1) {
23         // DAC Print
24         // Usart Print
25         if (system_print == 0) {
26             printf("U:%f,%f\r\n", Drive_AD2S.Electrical_Angle, Drive_hfi.theta_obs);
27         } else if (system_print == 1) {
28             printf("U:%f,%f\r\n", Drive_AD2S.Speed, Drive_hfi.speed_obs);
29         } else if (system_print == 2) {
30             printf("U:%f,%f\r\n", Drive_ish_filter.output, Drive_hfi.ish);
31         } else if (system_print == 3) {
32             printf("U:%f\r\n", Drive_hfi.u_h);
33         }
34     }
35 }
36
37 /**
38  * @brief   ADC Injected Channel interrupt Callback function
```

```

39  */
40  void HAL_ADCEx_InjectedConvCpltCallback(ADC_HandleTypeDef *hadc)
41  {
42      HAL_GPIO_WritePin(GPIOF, GPIO_PIN_4, 0); // Caculate running time
43
44      /*****
45       * @brief   Sample Calculate
46       */
47      hw_curr_sample(hadc);
48
49      // Angle and Speed Sample
50      AD2S1210_Angle_Get(); // Angle Sample
51      AD2S1210_Speed_Get(); // Speed Sample
52      /*****
53       * @brief   FOC Calculate
54       */
55      drive_foc_calc();
56      load_foc_calc();
57
58      /*****
59       * @brief   Voltage-Source Inverter Control
60       */
61      if (system_enable == 0) {
62          TIM8->CCR1 = 0;
63          TIM8->CCR2 = 0;
64          TIM8->CCR3 = 0;
65          TIM1->CCR1 = 0;
66          TIM1->CCR2 = 0;
67          TIM1->CCR3 = 0;
68      } else {
69          TIM8->CCR1 = Drive_duty_abc.dutya * TIM8->ARR;
70          TIM8->CCR2 = Drive_duty_abc.dutyb * TIM8->ARR;
71          TIM8->CCR3 = Drive_duty_abc.dutyc * TIM8->ARR;
72          TIM1->CCR1 = Load_duty_abc.dutya * TIM1->ARR;
73          TIM1->CCR2 = Load_duty_abc.dutyb * TIM1->ARR;
74          TIM1->CCR3 = Load_duty_abc.dutyc * TIM1->ARR;
75      }
76
77      /*****
78       * @brief   DAC Output
79       */
80      hw_dac_output();
81      HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_1, DAC_ALIGN_12B_R, system_dac_value1);
82      HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_2, DAC_ALIGN_12B_R, system_dac_value2);
83
84      HAL_GPIO_WritePin(GPIOF, GPIO_PIN_4, 1); // Caculate running time
85  }
86
87  /**
88   * @brief   Timer interrupt
89   */
90  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
91  {
92      if (htim->Instance == TIM8) {
93      }
94  }

```

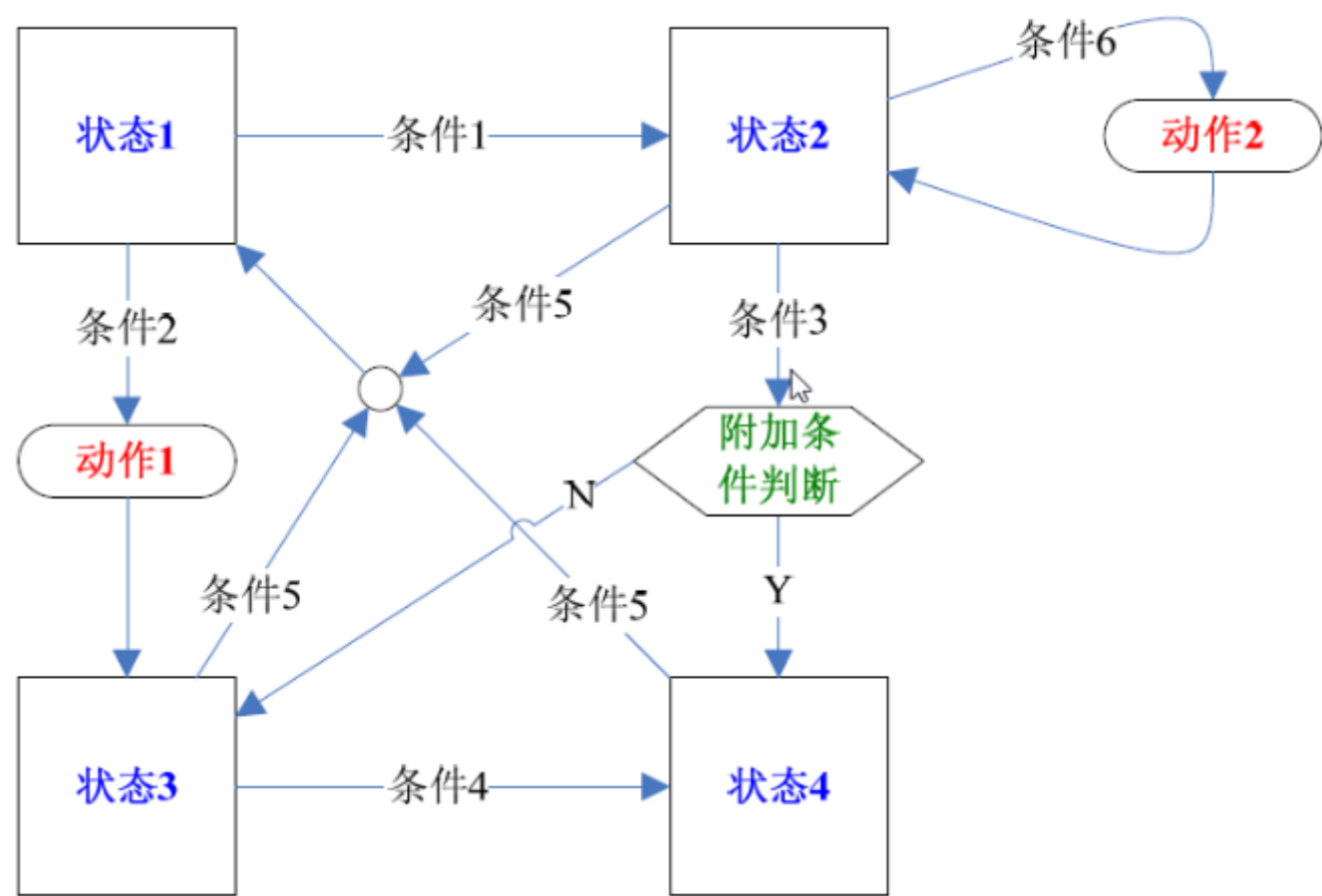
- `usermain.c` 文件和 CubeMX 生成的 `main.c` 文件地位一致，通常 `main.c` 文件执行完外设初始化后，就会直接进入 `usermain.c` 内的 `usermain()` 函数，也就是说 `usermain()` 函数是真正的 main 函数。这样可以避免 CubeMX 生成代码导致的可读性低。
- `usermain.c` 文件只放入系统最上层的线程(状态机/通信控制/中断线程)，为保证可读性(`usermain.c` 文件降低行数)，笔者更习惯于列出这些线程后对内部的内容进行二次封装。
- `usermain.h` 文件包含了所有头文件。

1. 中断线程可以分为快中断线程和慢中断线程，这取决于所需的控制频率和采样频率。

2. 控制算法应当精简，**中断占用率不能太高**，不然会死机。
3. 状态机可以采用 `switch-case` 架构：

```
1 switch(状态变量):
2     case 状态1:
3         // 设置该状态下启动的线程/指令变化
4         // 设置该状态下的状态转换条件
5         break;
6     case 状态2:
7         ...
8     default:
9         // 默认状态
10        break;
```

状态机可以控制指令的变化/子线程的解挂和挂起。在设计状态机前，可以**绘制状态转换图**帮助明确转换条件和每个状态下的动作。



4. **全局变量**可以在应用程序逻辑中出现。